

FitBuddy – AI Fitness Plan Generator

Overall System Architecture & Complete Source Code Implementation Report

PROJECT TITLE	AI ENGINE	AUTHOR / LEAD	TECH STACK
FitBuddy Generator	Google Gemini API	Ravi-teja-777	Python / Flask / JS

1 Project Overview & Ideation

FitBuddy addresses the limitation of generic fitness apps by taking user parameters (age, fitness goal, dietary type, equipment access, workout split preference) and passing them into **Google Gemini Models** to instantly synthesize personalized, safe, and actionable workout and diet blueprints.

2 System Architecture & Data Flow

Frontend Tier	Backend & LLM Tier
Responsive web portal providing intuitive parameter forms, async request handlers, and structured display panels for generated exercise/diet splits.	Python Flask backend handling request validation, structured prompt synthesis, Gemini API key authentication, and structured JSON parsing.

3 Core Backend Implementation (Python / Flask)

Below is the main server implementation handling user requests and communicating with the Gemini API:

```
app.py – Flask Server & Gemini SDK Integration

import os from flask import Flask, render_template, request, jsonify import google.generativeai as genai app = Flask(__name__) # Configure Google Gemini API Key GENAI_API_KEY = os.getenv("GENAI_API_KEY", "YOUR_GEMINI_API_KEY") genai.configure(api_key=GENAI_API_KEY) # Initialize Gemini Model model = genai.GenerativeModel('gemini-1.5-flash') @app.route('/') def index(): return render_template('index.html') @app.route('/generate-plan', methods=['POST']) def generate_plan(): data = request.json age = data.get('age', '25') goal = data.get('goal', 'Muscle Gain') diet = data.get('diet', 'Non-Vegetarian') equipment = data.get('equipment', 'Dumbbells & Bodyweight') days = data.get('days', '4') # Construct strict prompt for JSON structure prompt = f""" You are an expert certified personal trainer and sports nutritionist. Create a personalized {days}-day fitness and diet plan for a user with: - Age: {age} | Goal: {goal} | Diet: {diet} | Equipment: {equipment} Return response strictly as a JSON object with this exact key structure: {{ "plan_title": "String", "daily_calories": "String", "macro_split": {{ "protein": "String", "carbs": "String", "fats": "String" }}, "weekly_schedule": [ {{ "day": "Day 1", "focus": "String", "exercises": [{{ "name": "String", "sets": "String", "reps": "String" }}, {"nutrition_tip": "String" }} ] }} ] }} """ try: response = model.generate_content(prompt, generation_config={"response_mime_type": "application/json"}) return jsonify({"success": True, "data": response.text}) except Exception as e: return jsonify({"success": False, "error": str(e)}), 500 if __name__ == '__main__': app.run(debug=True, port=5000)
```

4 Frontend Interface & API Controller Code

The web client posts structured form data to the backend endpoint and dynamically renders the LLM response.

static/js/main.js – Client-Side Controller

```
async function generateFitnessPlan() { const formData = { age: document.getElementById('age').value, goal: document.getElementById('goal').value, diet: document.getElementById('diet').value, equipment: document.getElementById('equipment').value, days: document.getElementById('days').value }; const response = await fetch('/generate-plan', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(formData) }); const result = await response.json(); if (result.success) { const plan = JSON.parse(result.data); renderOutput(plan); } else { alert("Error generating plan: " + result.error); } } function renderOutput(plan) { let html = `<h3>${plan.plan_title}</h3>`; html += `<p><strong>Target Calories:</strong> ${plan.daily_calories}</p>`; plan.weekly_schedule.forEach(day => { html += `<div class="day-card"> <h4>${day.day}: ${day.focus}</h4> <ul>` + day.exercises.map(ex => `<li>${ex.name} - ${ex.sets} sets x ${ex.reps}</li>`).join('') + `</ul>` </div>`; }); document.getElementById('plan-results').innerHTML = html; }
```

5 Quality Assurance & SDLC Folder Mapping

This repository aligns with standard SDLC deliverables across the 8 project directories:

SDLC Directory Folder	Primary Artifact / Code File	Verification Standard
1. Brainstorming & Ideation	concept_doc.md	Problem formulation & AI fit
2. Requirement Analysis	functional_reqs.json	Parameter constraints & goals
3. Project Design Phase	architecture_diagram.png	Data flow & API topology
4. Project Planning Phase	milestones.md	Delivery roadmap & sprint plan
5. Project Development Phase	app.py, index.html, main.js	Full Implementation
6. Project Testing	test_api.py	JSON Schema & latency testing
7. Project Documentation	API_DOCS.md	Setup & usage parameters
8. Project Demonstration	demo_video.mp4 / screenshots/	End-to-end execution walkthrough

6 Execution Instructions

- **Install Dependencies:** `pip install flask google-generativeai`
- **Environment Setup:** Set `export GEMINI_API_KEY="your_api_key_here"`
- **Run Application:** Execute `python app.py` and access `http://localhost:5000`

